NAME(S):

Lists

Begin the following as paired-programming in class, but be sure to individually finish any problems you don't get to (hand in written answers of **0.25 engagement points**, check website for due date)

Learning goals / relevent topics

- String methods (Runestone Ch. 7)
- List methods (Runestone Ch. 9)

List Practice Problems

- Remember to test all the functions you write by calling them to make sure they are both *syntactically valid* and *semantically correct*.

1. Define a function called `print_negatives` that takes a list of numbers as a parameter and prints out any negative numbers in that list.
2. Define a function called `any_greater` that takes a list of numbers and a single number as parameters and checks whether any numbers in the list are greater than the given number.
3. Define a function called `average` that takes a list of numbers and computes the mean.
4. Define a function called `standard_deviation` that takes a list of numbers and calculates the standard deviation. Note: standard deviation is the square root of variance.
5. Define a function called `variance` that takes a list of numbers and calculates the population variance of that list of numbers.

- Note: Population variance is the mean of the squared deviations of the numbers from the mean of the list. That is, for each number in the list, you have to calculate the difference between that number and the mean of the list, then square that difference. The population variance is the mean of all those squared differences.

6. Define a function called `count` that takes a list and a second value as parameters and counts how often the given second value occurs in the given list.

- Try this problem using built in list methods and then without using the built in list methods.

7. Define a function called `word_lengths` that takes a list of strings as a parameter and returns a new list which represents the lengths of all strings in the given list. For example, given the list

```
["dolphin", "turtle", "fish", "sealion", "whale"]
```

the function should return `[7, 6, 4, 7, 5]`.

- Hint: the built-in len function may be useful here.

8. Define a function called `word_counts` that takes two lists and returns a list that shows how often the elements in the second list occur in the first list. For example, given the two lists:

```
['Bilbo', 'was', 'very', 'rich', 'and', 'very', 'peculiar,', 'and', 'had', 'been', 'the',
'wonder', 'of', 'the', 'Shire', 'for', 'sixty', 'years,', 'ever', 'since', 'his', 'remarkable',
'disappearance', 'and', 'unexpected', 'return.', 'The', 'riches', 'he', 'had', 'brought',
'back', 'from', 'his', 'travels', 'had', 'now', 'become', 'a', 'local', 'legend,', 'and',
'it', 'was', 'popularly', 'believed,', 'whatever', 'the', 'old', 'folk', 'might', 'say,',
'that', 'the', 'Hill', 'at', 'Bag', 'End', 'was', 'full', 'of', 'tunnels', 'stuffed', 'with',
'treasure.', 'And', 'if', 'that', 'was', 'not', 'enough', 'for', 'fame,', 'there', 'was',
'also', 'his', 'prolonged', 'vigour', 'to', 'marvel', 'at.', 'Time', 'wore', 'on,', 'but',
'it', 'seemed', 'to', 'have', 'little', 'effect', 'on', 'Mr.', 'Baggins.', 'At', 'ninety',
'he', 'was', 'much', 'the', 'same', 'as', 'at', 'fifty.', 'At', 'ninety-nine', 'they',
'began', 'to', 'call', 'him', 'well-preserved,', 'but', 'unchanged', 'would', 'have', 'been',
'nearer', 'the', 'mark.', 'There', 'were', 'some','that', 'shook', 'their', 'heads', 'and',
'thought', 'this', 'was', 'too', 'much', 'of', 'a', 'good', 'thing;', 'it', 'seemed', 'unfair',
'that', 'anyone', 'should', 'possess', '(apparently)', 'perpetual', 'youth', 'as', 'well',
'as', '(reputedly)', 'inexhaustible', 'wealth.']
```

and `['and', 'very', 'tunnels']`, the function should return `[5, 2, 1]`.

- Hint: it may be helpful to functions you've already defined.

Prompt: What are the differences between strings and lists in python?